

Gramática libre de contexto y Left Factoring

Compiladores - Equipo 07
Yañez Barajas Brandon - Model Designer
2 de octubre de 2026

1. Ejemplo seleccionado

Elegí la entrada del profesor `int a = 10;`. También consideré `int a;` para mostrar dos alternativas con un prefijo común. La CFG describe la secuencia de tokens: *id* representa un identificador y *num* una constante numérica. Ambos son terminales de esta gramática.

2. Gramática original

Defino $G_0 = (N_0, T, P_0, S)$, donde:

$$N_0 = \{S\}, \quad T = \{\text{int}, id, num, =, ;\}.$$

El símbolo inicial es S y las producciones son:

$$S \rightarrow \text{int } id = num ; \mid \text{int } id ;.$$

Cada producción tiene un único no terminal a la izquierda; por ello, la gramática es libre de contexto.

3. Factorización por la izquierda

Las alternativas comparten el prefijo `int id`. Separé este prefijo e introduje el no terminal R para representar el resto:

$$\begin{aligned} S &\rightarrow \text{int } id R, \\ R &\rightarrow = num ; \mid ;. \end{aligned}$$

La nueva gramática es $G_1 = (N_1, T, P_1, S)$, con $N_1 = \{S, R\}$ y P_1 formado por las reglas anteriores. T y el símbolo inicial se conservan. $R \rightarrow ;$ no es una producción vacía: genera el punto y coma de la declaración sin inicialización.

4. Derivaciones

Con inicialización:

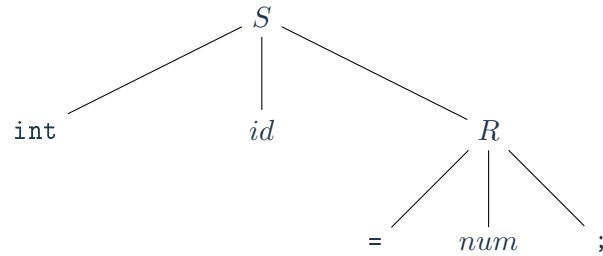
$$S \Rightarrow \text{int } id R \Rightarrow \text{int } id = num ;.$$

Sin inicialización:

$$S \Rightarrow \text{int } id R \Rightarrow \text{int } id ;.$$

El lexer establece que `a` corresponde a *id* y `10` a *num*. No se agregan reglas $id \rightarrow a$ ni $num \rightarrow 10$: la gramática opera sobre categorías de tokens y terminales específicos.

5. Árbol de derivación



Las hojas de izquierda a derecha son `int`, `id`, `=`, `num`, `;`. Coinciden con la secuencia de la declaración seleccionada.

6. Comparación y conservación del lenguaje

La gramática original genera las dos estructuras siguientes:

$$L(G_0) = \{\text{int } id = num ;, \text{ int } id ;\}.$$

En la gramática factorizada, S genera el prefijo común y R permite exactamente los dos sufijos originales. Por lo tanto, $L(G_0) = L(G_1)$. La transformación reorganiza las producciones sin añadir ni eliminar estructuras.

7. Relación con el proyecto

El lexer se implementará mediante expresiones regulares. La CFG describe el orden de los tokens y no depende del mecanismo utilizado para reconocerlos. Los AFND y AFD se conservan como documentación del modelo teórico.

Esta propuesta cubre declaraciones sencillas de tipo `int`; no constituye una gramática completa de C. La tarea presenta la gramática, su transformación y comprobación, sin implicar que el lexer tenga un parser o valide la sintaxis de las entradas.